



Machine Learning

Project Presentation [a.a. 24/25]

Intro & Objectives



My goal was to develop a fully parametrizable **neural network simulator** *from scratch*, capable of handling both classification and regression tasks. Additionally, I aimed to establish a systematic approach for model selection and assessment.

I implemented a fully connected, feedforward multi-layer perceptron trained via backpropagation and stochastic gradient descent, supporting optional momentum acceleration.

Method - Implementation



Libraries - The whole implementation of the MLP was made from scratch. I utilized some numerical, graphical, utility libraries:

- numpy, pandas - *numerical*
- matplotlib - *graphical*
- scikit-learn - *utility*: just to split dataset, generate combinations from a defined parameters grid and 1-of-k encoding
- joblib - *utility*: to parallelize hyperparameters search

Method - Implementation



Code [NN params] - The implementation is parametrizable in the:

- Number of layers, and number of units
- Weight Initialization: *He, Lecun, Glorot*
- Activation functions: *ReLU, Tanh, Linear, Sigmoid*
- Learning rate
- Learning rate decay (type and rate): *Linear, Exponential, None*
- Regularization (type and lambda): *L1, L2, None*
- Error functions: *MSE, Binary Crossentropy*
- Momentum (type and alpha): *Momentum, None*

Method - Implementation



Code [NN training] - `train()` it's a method provided by the MLP class, we can specify:

- Number of epochs
- Batch size (thus, abstracting *Mini Batch*, *Online*, *Full Batch*)
- Validation set (if provided, the networks tracks the validation error and accuracy over the epochs. The VL set is also needed for early stopping)
- Early stopping (*whether it must be done*, with what *patience* and for which *minimum delta*)

Code [model selection & more] - w.r.t. model selection, I created the code for:

- Conduct a K-Fold Cross validation
- Run a Grid Search (including also cross validation inside)

Monk's - Results (avg. on 3 trials)

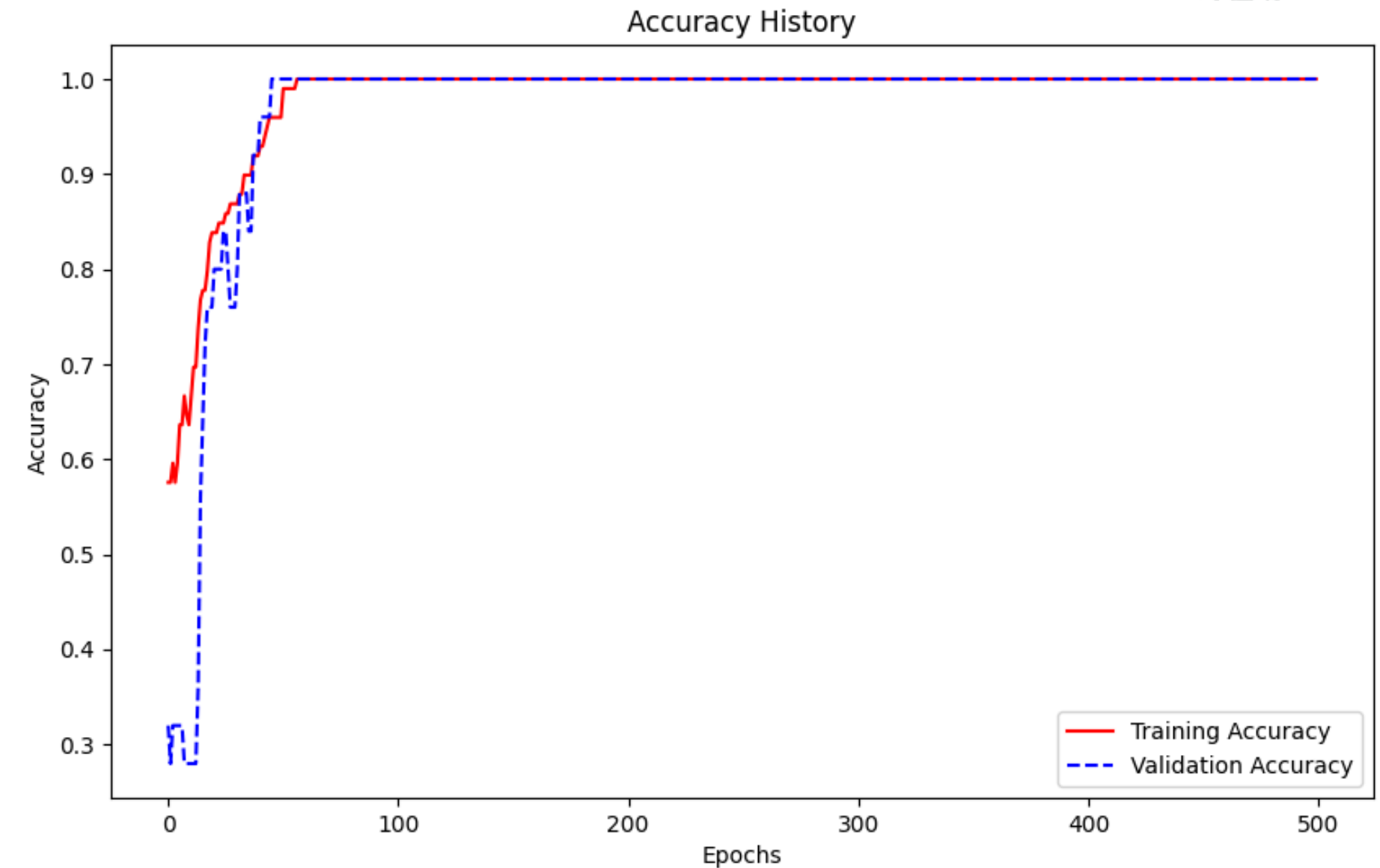
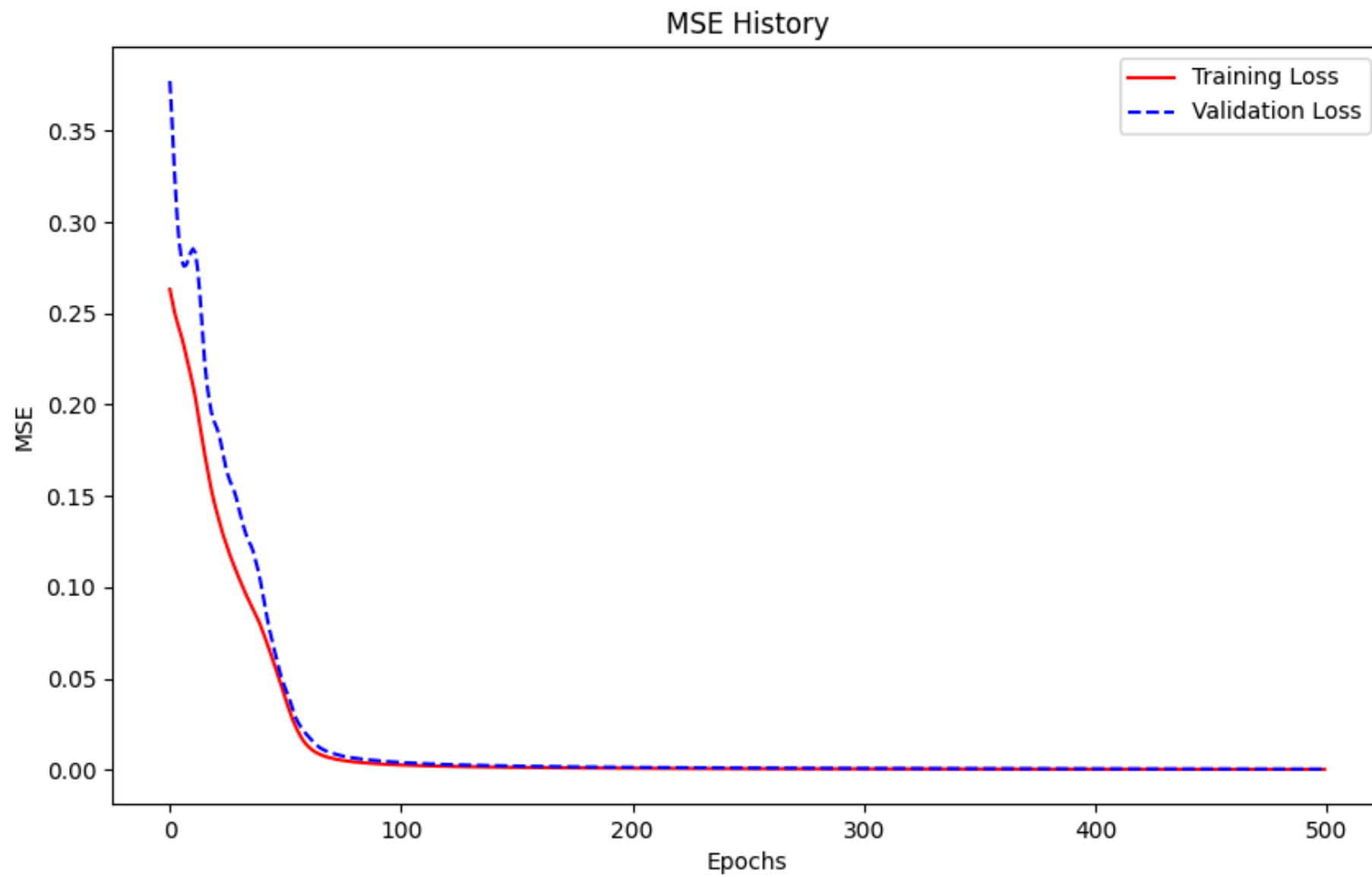


Task	(1 Hidden layer) # Hidden Units	η	Momentum (Type/ α)	Weight Init	Reg. (Type/ λ)	MSE (VL/TS)	Accuracy (VL/TS) (%)
MONK 1	3	0.81	Momentum / 0.8	Glorot	None / 0	0.0079 / 0.0004	99 / 100
MONK 2	5	0.76	Momentum / 0.8	Base	None / 0	0.0003 / 0.0006	100 / 100
MONK 3	4	0.58	Momentum / 0.7	He	L2 / 0.0001	0.0644 / 0.0156	92 / 95

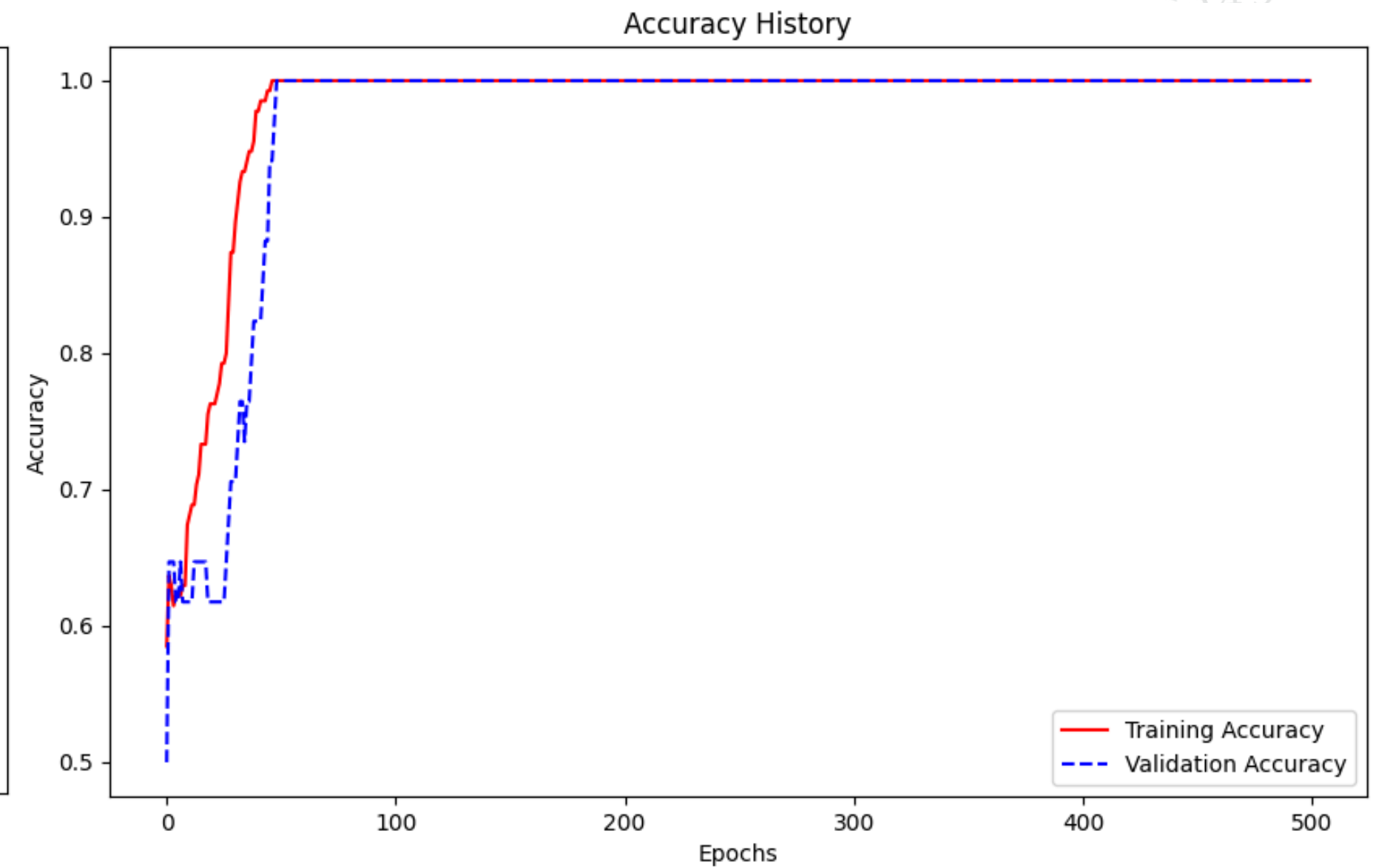
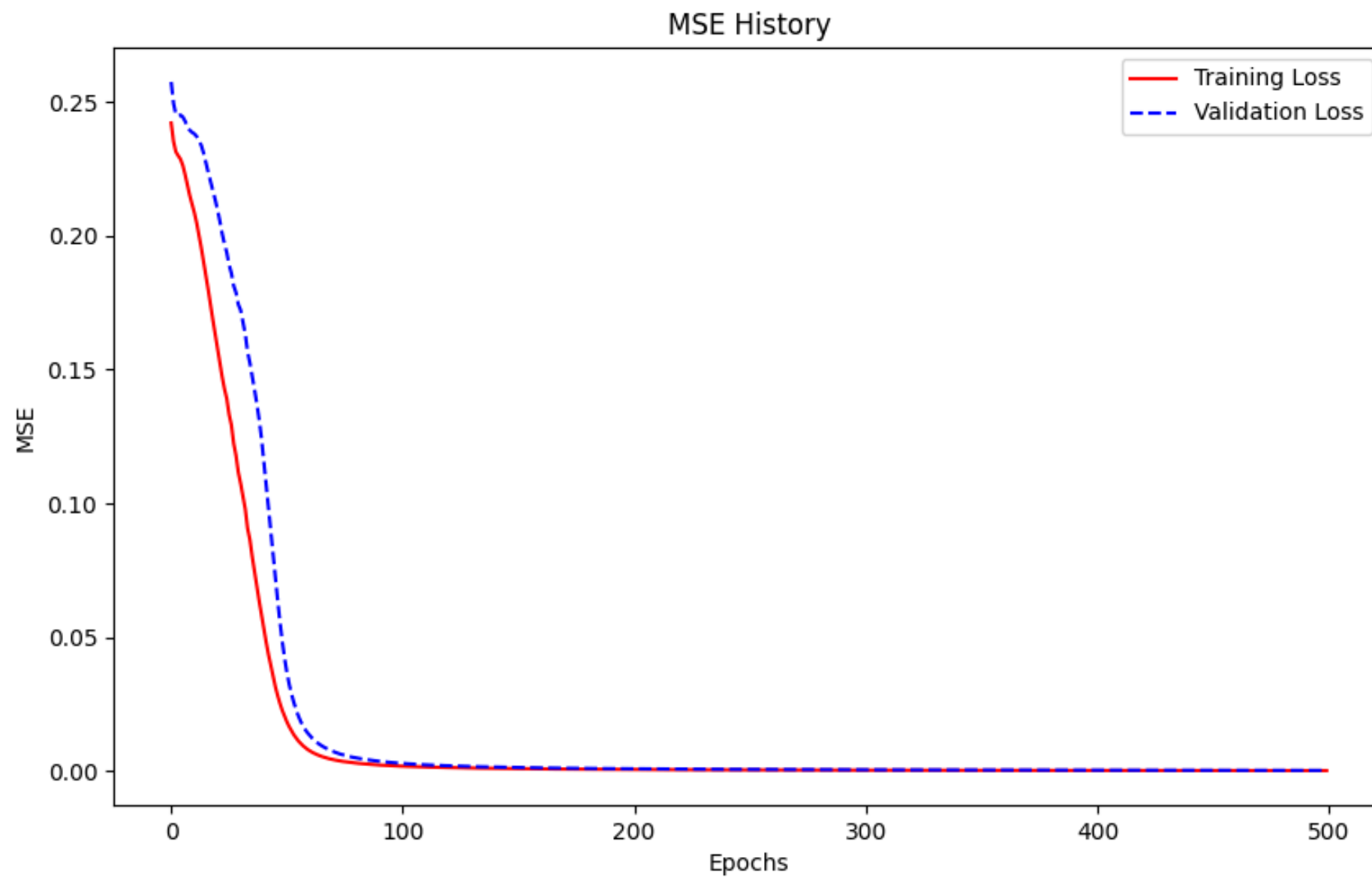
For each task:

- One-hot encoding was used (*see appendix for details*).
- The batch size was set to the full dataset.
- The best model was selected based on the lowest mean and standard deviation of the Mean Squared Error (MSE) across k-folds from the grid search results (*see appendix for details*).

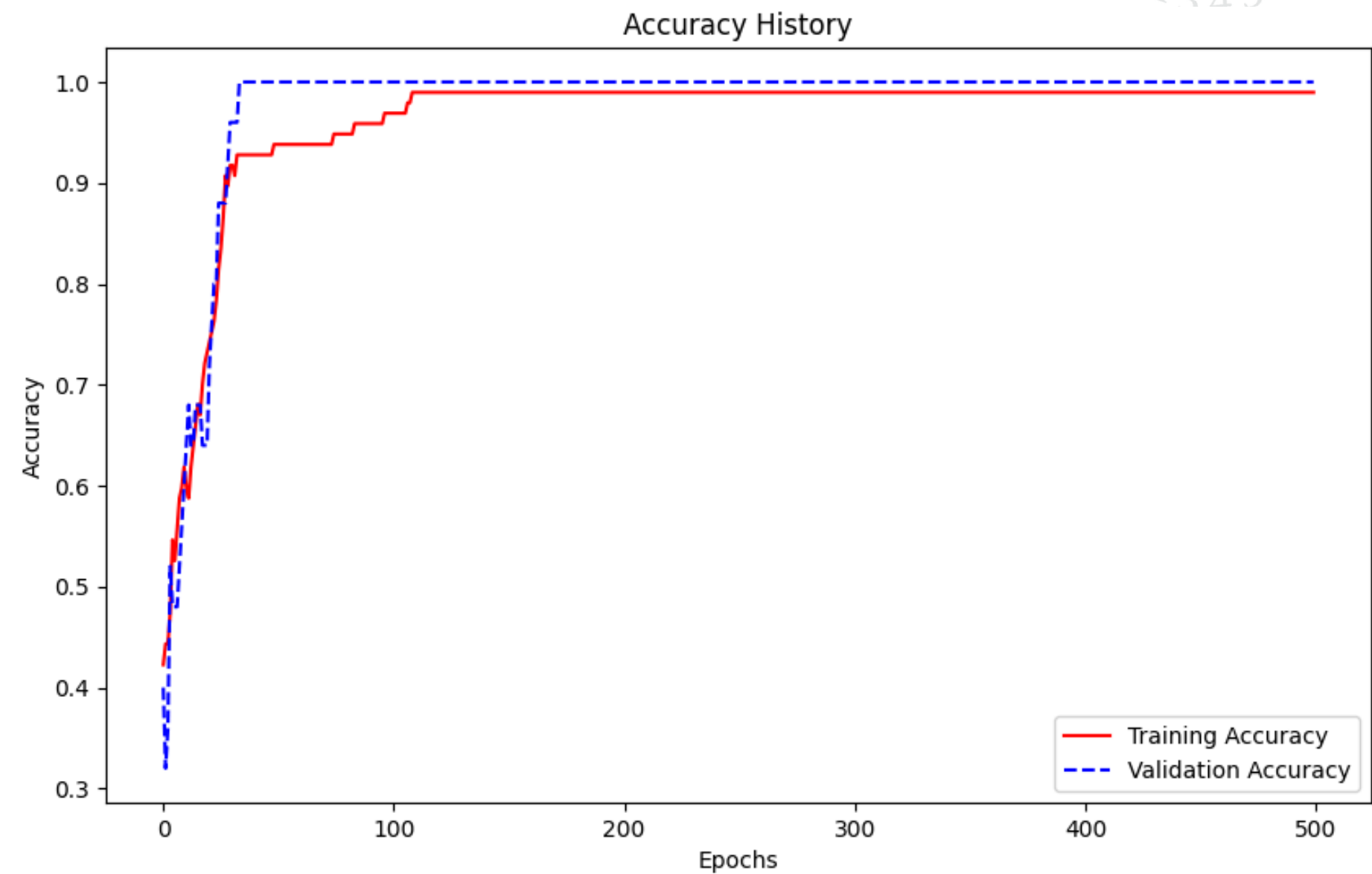
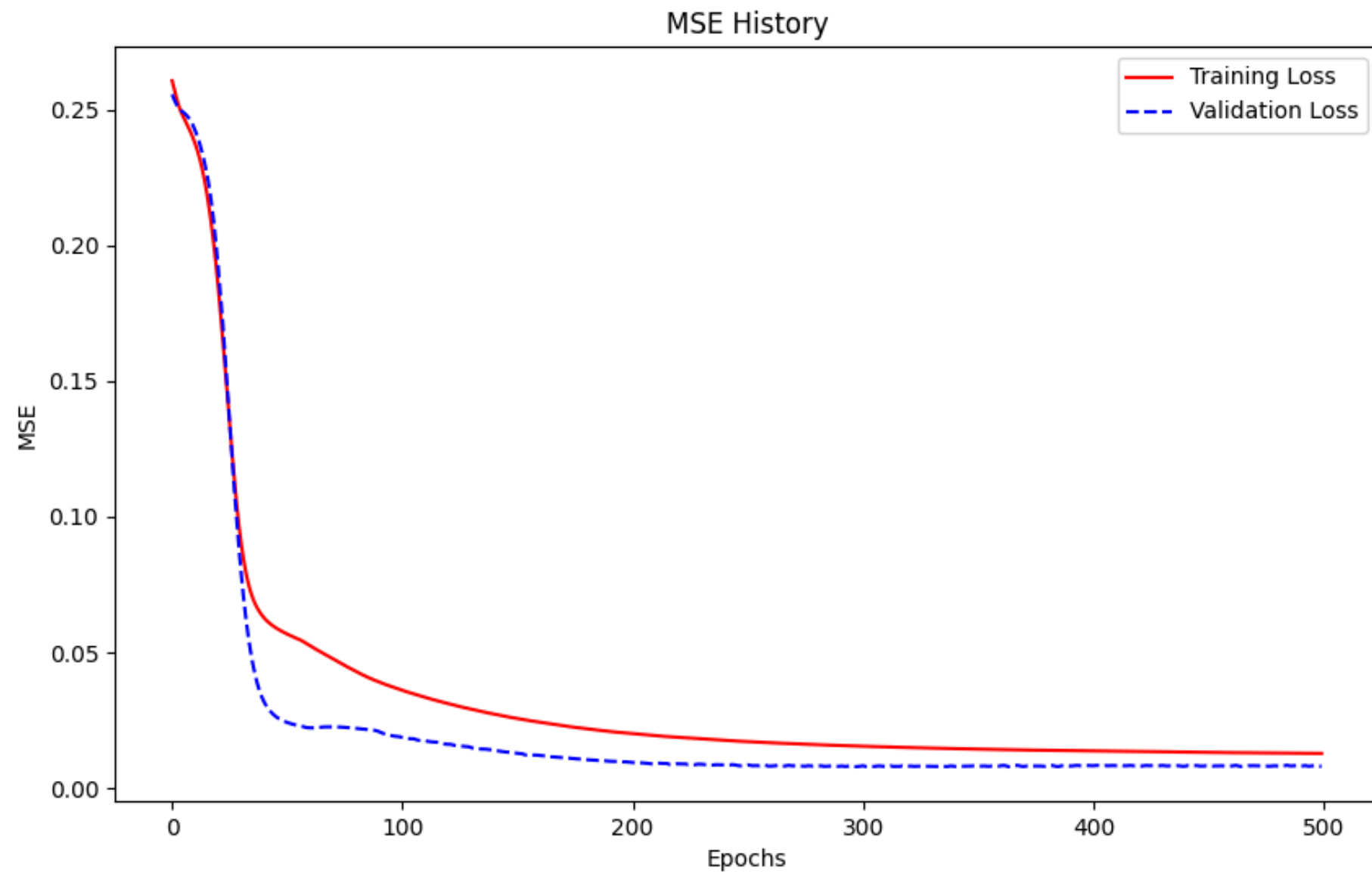
Monk's 1 - Plots



Monk's 2 - Plots



Monk's 3 (reg) - Plots



CUP - Validation Schema



Dataset was splitted into:

5-Fold CV Development Set 80%	Hold Out Test Set 20%
---	---------------------------------

A **retraining** was made on the whole dev set for model assessment and **another retraining** was made before the predictions on the blind set.

CUP - Strategy



1. Long **screening phase** to check hyperparameters effects
 2. **Coarse**-grained grid search
 3. **Fine**-grained grid search
- } $\approx$ (2.5 hours on Apple chip M3 - 11 core)
parallel search
- Models were **trained** with **MSE**, and **evaluated** considering the mean & std.dev. **MEE** over the 5 folds.
 - **Early stopping** with patience = 10

CUP - Screening



Evaluations and comparisons tried:

- *# Hidden Layers & # Hidden Units*
 - Combination with too few units seems to be unable to converge well. One layer seems to be enough.
 - *Batch size & Learning rate*
 - The dataset seems to require low learning rate [O(0.001)] to converge well. As seen in theory, higher batch size requires higher learning rate.
 - *Batch size & Lambdas*
 - Low batch sizes makes the learning curve very unsmooth! Momentum helps, but the plot definitely tells us that 16 is a nice bs. (it will be confirmed also by the coarse grid search)
 - *Hidden layers & Hidden activation functions*
 - *Learning rate & Lambdas*
 - *Learning rate & Decay rates & Decay type*
 - *Learning rate & Alphas & Momentum type*
 - *...many more*
- (check appendix for plots)

CUP - Coarse Grid Search



Params Grid $\approx$ **300k** combination of parameters

```
coarse_param_grid = {
    "learning_rate": [0.001, 0.0001, 0.00001],
    "layers": [
        [12, 10, 3],
        [12, 15, 3],
        [12, 20, 3]
    ],
    "decay_rate": [0, 1e-1, 1e-2, 1e-04, 1e-06],
    "lambda_reg": [0, 1e-7, 1e-5, 1e-3, 1e-1],
    "momentum_alpha": [0, 0.8, 0.7, 0.6],
    "momentum_type": ["none", "momentum"],
    "weight_init": ["glorot", "base", "he"],
    "activation_function_names": [
        ["relu", "linear"],
        ["tanh", "linear"],
    ],
    "reg_type": ["none", "l1", "l2"],
    "lr_decay_type": ["none", "linear", "exponential"],
}
```

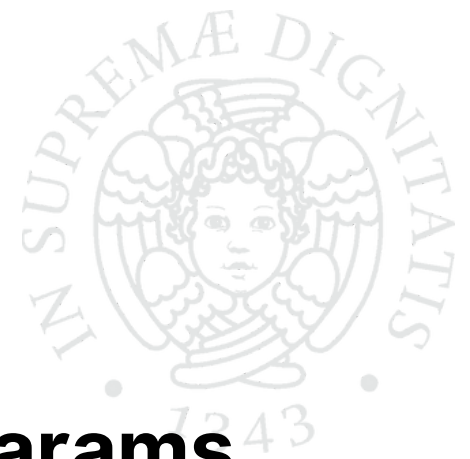
```
BATCH_SIZE = [16, 32, 64]
```

Best Result

```
"reg_type" : "l1",
"lambda_reg" : 1e-05,
"momentum_alpha" : 0.8,
"momentum_type" : "momentum",
"weight_init" : "base",
"activation_function_names" : ["relu", "linear"],
"loss_function_name" : "mse",
"task" : "regression",
"layers" : [12, 20, 3],
"learning_rate" : 0.001,
"decay_rate" : 0,
"lr_decay_type" : "none",
"batch_size" : 16
```

Average MEE: 0.84

CUP - Which values to retain? 🤔



I retained some values from the coarse search checking the **frequencies** of the **params** on the **top-10** results (score = weighted position, normalized) (*details in the appendix*):

Hyperparameter	Value	Score
learning_rate	0.001	1.0000
layers	[12, 20, 3]	0.9171
	[12, 15, 3]	0.0829
decay_rate	0	0.5700
	0.0001	0.4300
lambda_reg	1e-07	0.3414
	0.001	0.2987

Hyperparameter	Value	Score
lambda_reg	1e-05	0.3029
	0	0.0569
momentum_alpha	0.8	0.8748
	0.7	0.0683
	0.6	0.0569
momentum_type	momentum	1.0000
weight_init	glorot	0.6288
	base	0.2845

Hyperparameter	Value	Score
weight_init	he	0.0867
activation_function_names	['relu', 'linear']	1.0000
reg_type	l2	0.6889
	l1	0.2542
	none	0.0569
lr_decay_type	none	0.5700
	exponential	0.2195
	linear	0.2105
batch_size	16	1.0000

CUP - Fine Grid Search



Params Grid $\approx$ **9k** combination of parameters

Best Result (follows the top-3 table)

```
fine_param_grid = {
    "learning_rate": np.arange(0.0001, 0.001, 0.00005),
    "layers": [[12, 20, 3]],
    "decay_rate": [0]+np.arange(0.00001, 0.0001, 0.00001).tolist(),
    "lambda_reg": [1/10**i for i in (np.arange(1,6,1).tolist())],
    "momentum_alpha": [0.75, 0.8, 0.82],
    "momentum_type": ["momentum"],
    "weight_init": ["glorot", "he"],
    "activation_function_names": ["relu", "linear"],
    "reg_type": ["l2"],
    "lr_decay_type": ["none", "linear"]
}
```

```
"reg_type": "l2",
"lambda_reg": 0.001,
"momentum_alpha": 0.82,
"momentum_type": "momentum",
"weight_init": "he",
"activation_function_names": ["relu", "linear"],
"loss_function_name": "mse",
"task": "regression",
"layers": [12, 20, 3],
"learning_rate": 0.00085000000000000002,
"decay_rate": 8e-05,
"lr_decay_type": "linear",
```


CUP - Fine Grid Search



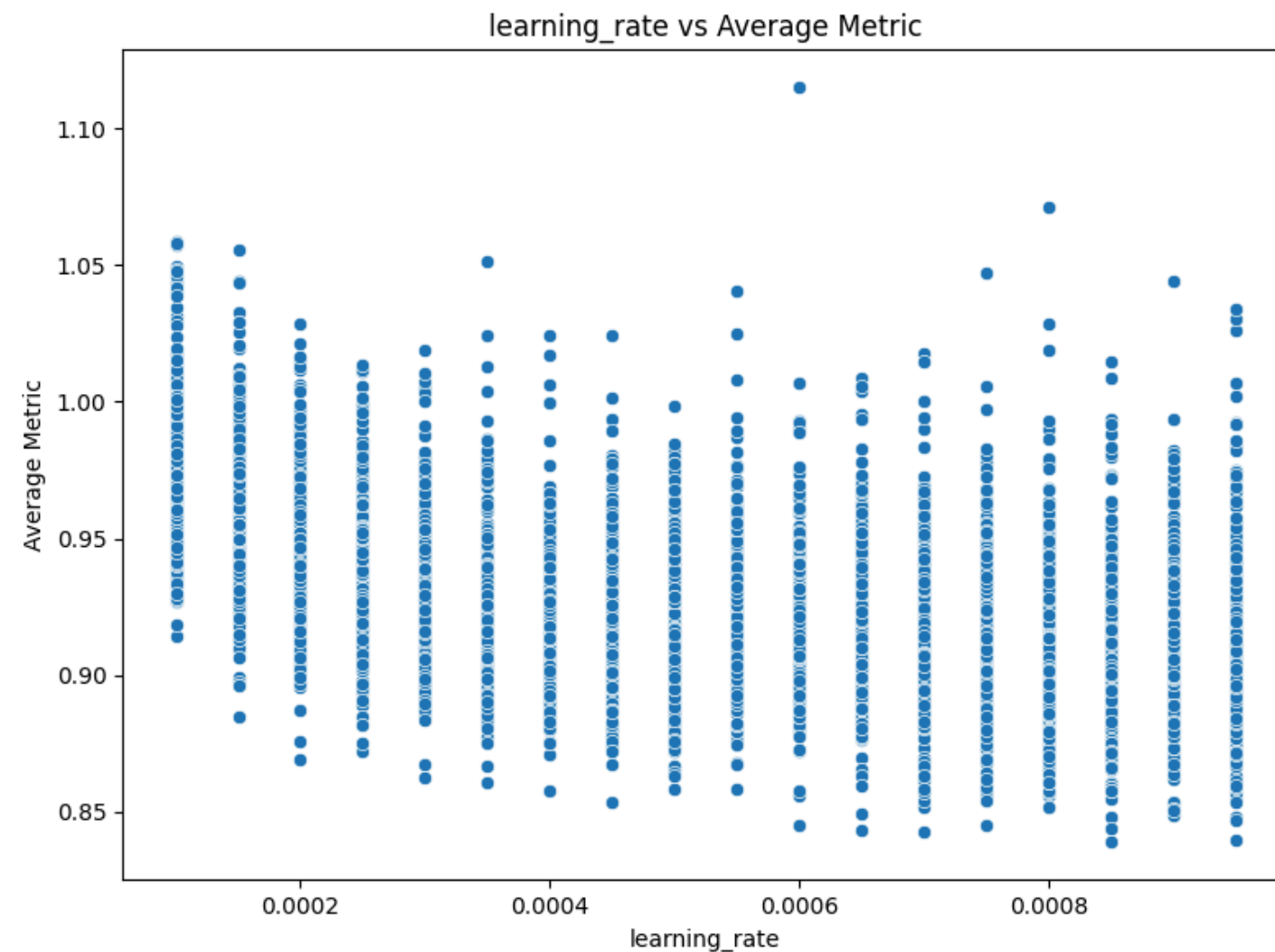
Best 3 results

η	Reg. Type / λ	LR (η) Decay Rate / Type	α (momentum)	(1 Hidden layer) # Hidden Units	Activation Function	Training MEE	Validation MEE
0.00085	L2 / 0.001	8e-05 / Linear	0.82	20	ReLU	0.827	0.836
0.00095	L2 / 0.001	1e-05 / Linear	0.82	20	ReLU	0.839	0.893
0.00070	L2 / 0.0001	3e-05 / Linear	0.82	20	ReLU	0.879	0.924

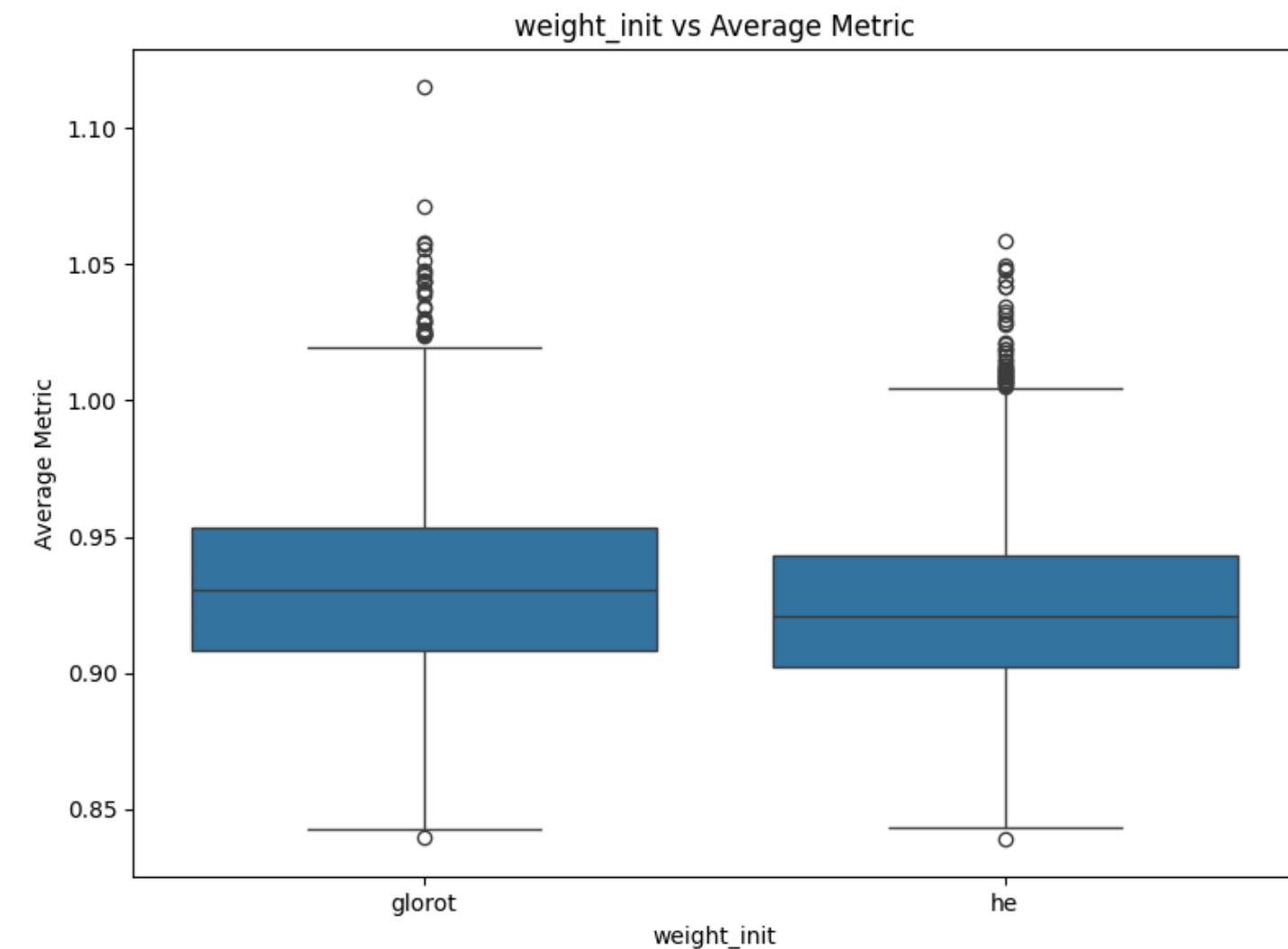
CUP - Intermezzo



Some hyperparameters trend:



Values around 0.0008 unlock lower mean error



On average, *He initialization* is a bit better both for standard deviation and for mean of error

CUP - Final Model



The Final Model has been selected considering the lowest **Average and Standard deviation of MEE** over the folds

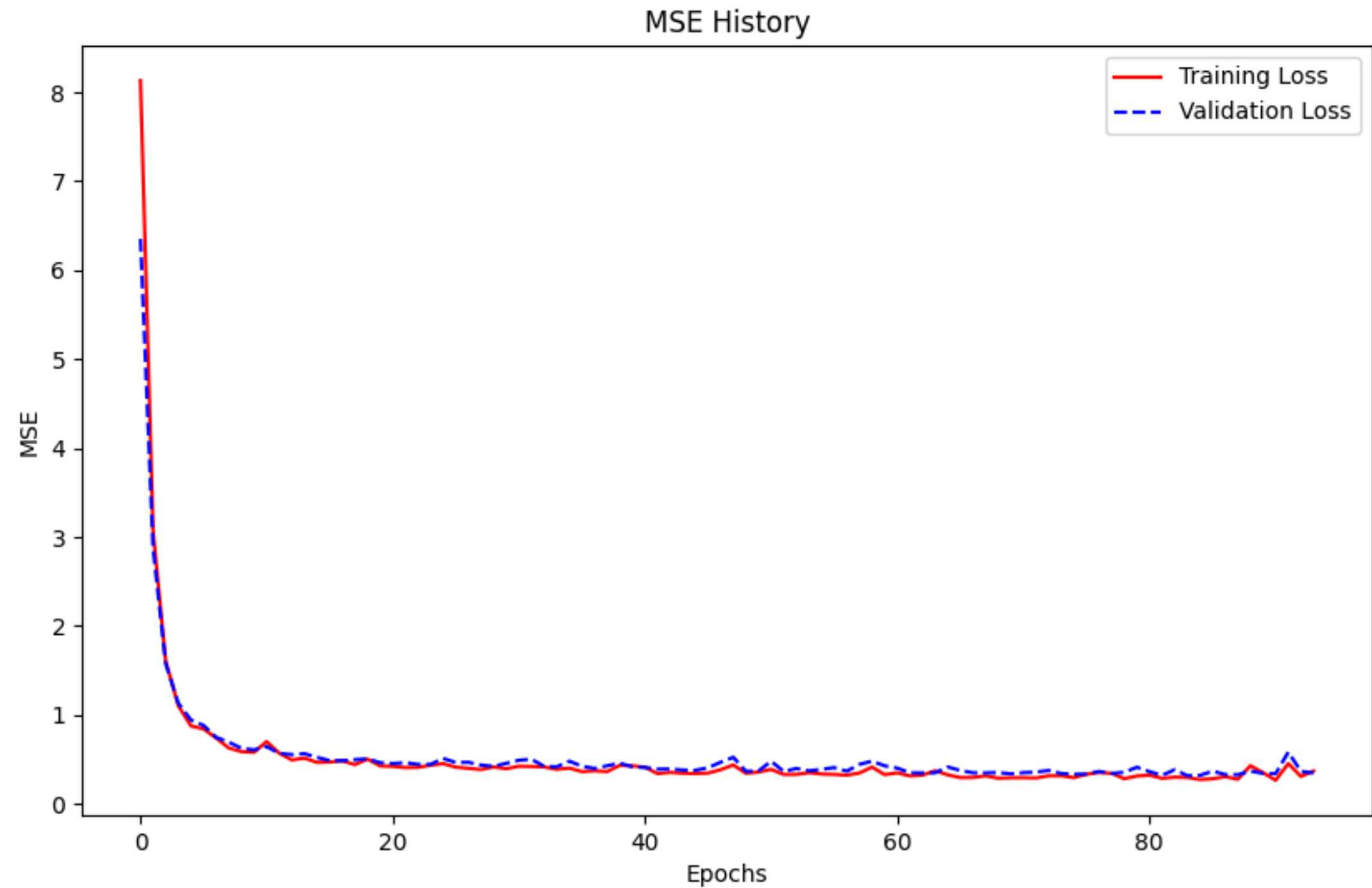
```
# == Neural Network Summary: == #  
    Task: Regression  
    Error Function: Mse  
    Regularization: L2 (lambda=0.001)  
    Learning Rate: 0.00085  
    Learning Rate Decay: Linear (decay_rate=8e-05)  
    Momentum: Momentum (alpha=0.82)  
    Weight Initialization: He  
    Architecture: [12, 20, 3]  
    Activation Functions: ['relu', 'linear']  
# ===== #
```

CUP - Final Model



Plots and results

MEE TR	MEE VL	MEE TS
0.826	0.836	0.745



CUP - Discussion



- For the CUP dataset, a lower learning rate is necessary when using MSE as the error function.
- The model is highly sensitive to hyperparameters, making an initial screening phase very useful.
- Early stopping proved to be highly effective in mitigating overfitting and enhancing generalization.
- Momentum methods help smooth the learning curve.
- It's interesting to notice that the top three configurations all include regularization, momentum, and learning rate decay.

Conclusions



Implementing the neural network was a deeply rewarding experience. Though debugging a model from scratch can be maddening - why isn't the loss decreasing? Why are the weights exploding? - **each challenge strengthens fundamental understanding**. Every misstep refines intuition, reveals hidden assumptions, and sharpens analytical thinking.

Team name: ***LambdaX***

I agree to the disclosure and publication of my name, and of the results with preliminary and final ranking.

References



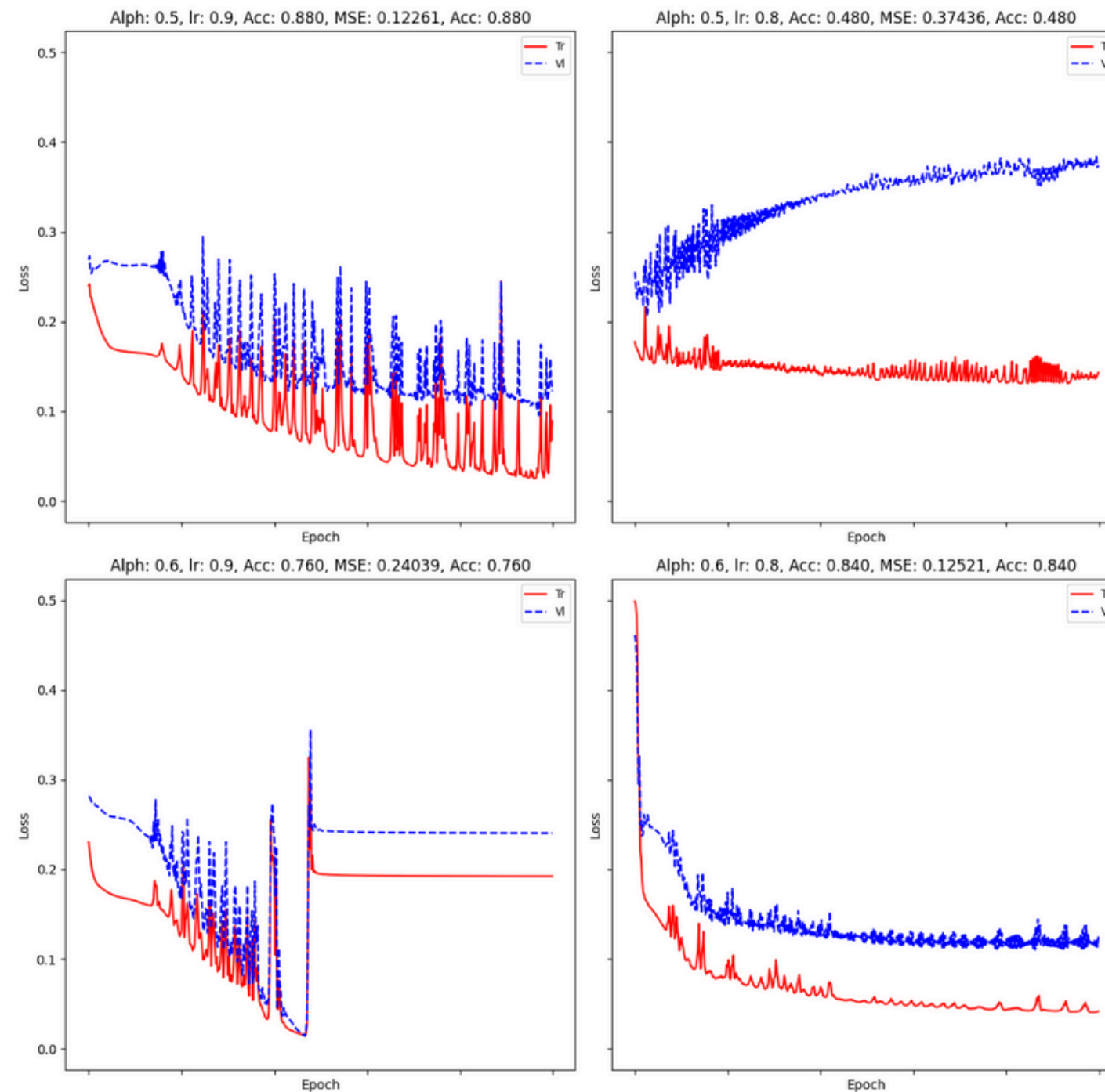
- [1] Bengio, Y. & Glorot, X.. (2010). Understanding the difficulty of training deep feed forward neural networks. International Conference on Artificial Intelligence and Statistics. 249-256.
- [2] LeCun, Y.A. et al. (2012). *Chapter 1. - Efficient BackProp - 1.4.6 Initializing the Weights*. In G. Montavon, G.B. Orr, & K.-R. Müller (Eds.), *NN: Tricks of the Trade* (2nd ed., Vol. 7700, pp. 28-29). Springer Heidelberg
- [3] Prechelt, Lutz. (2000). *Early Stopping - But When?*. Lecture Notes in Computer Science. 10.1007/3-540-49430-8_3.
- [4] Thrun, S. & Bala, Jerzy & Bloedorn, Eric & Bratko, Ivan & Cheng, J. & De Jong, Kenneth & Džeroski, Sašo & Fahlman, Scott & Fisher, Doug & Hamann, R. & Kaufman, Kenneth & Keller, S. & Kononenko, I. & Kreuziger, J. & Mitchell, T. & Pachowicz, P. & Reich, Yoram & Vafaie, Haleh & Wnek, J.. (1991). *The MONK's Problems: A Performance Comparison of Different Learning Algorithms*.

Appendix: One Hot Encoding for Monk's

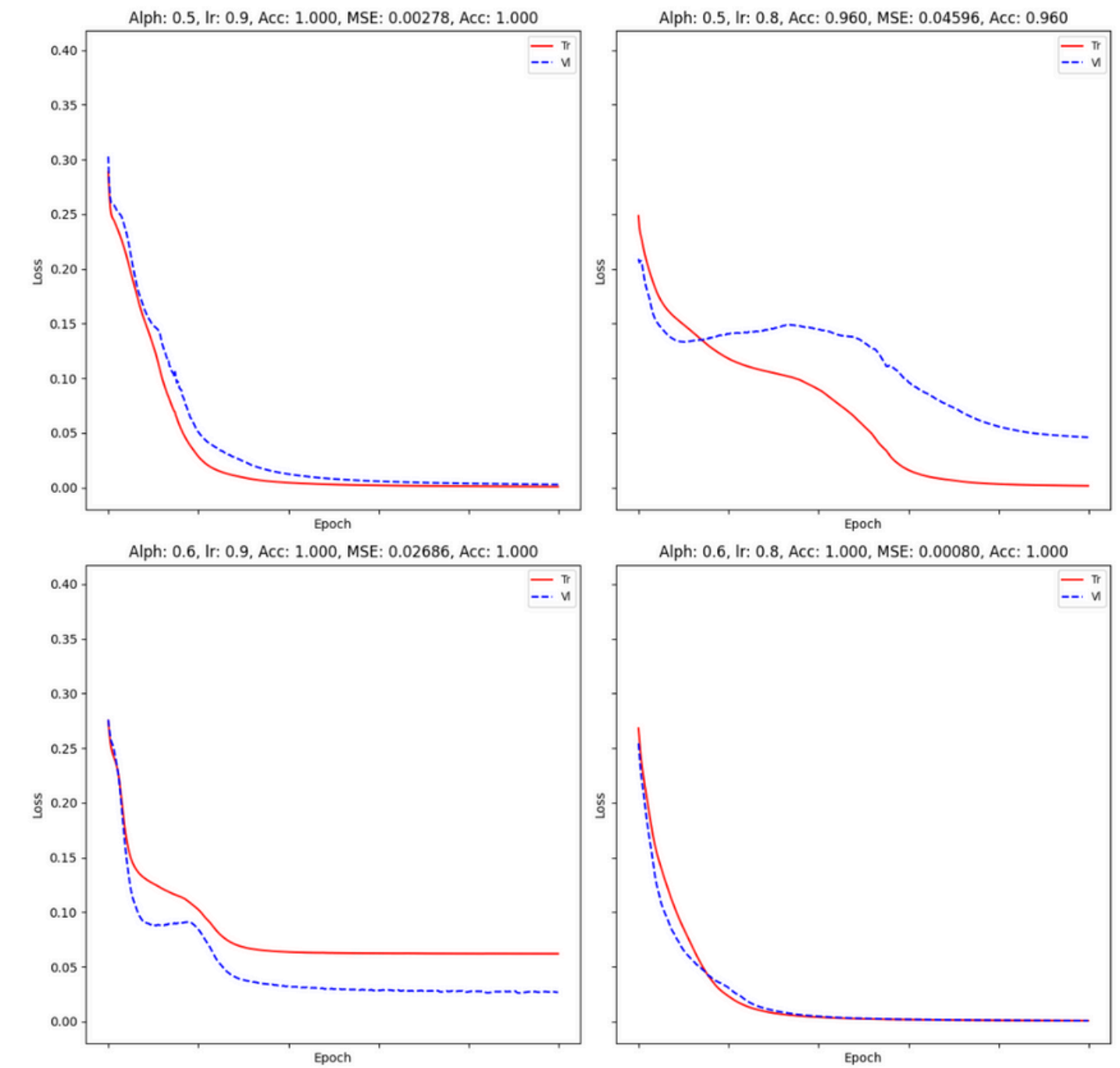


The same
params
combo
with /
without
1-hot
encoding

Without



With 1-of-k enc.



Appendix: Hyperparams Grid for Monk's Task



Parameters grid for First and Second Monk task

Parameters grid for Third Monk task

```
param_grid_1 = {
    "learning_rate": np.arange(0.1, 0.9, 0.01),
    "layers": [ [17, i, 1] for i in [3, 4, 5] ],
    "decay_rate": [0],
    "lambda_reg": [0, ],
    "momentum_alpha": [0] + np.arange(0.5, 0.9, 0.1).tolist(),
    "momentum_type": ["none", "momentum"],
    "weight_init": ["glorot", "base", "he"],
    "activation_function_names": [
        [i, "sigmoid"] for i in ["relu", "tanh", "sigmoid"]
    ],
    "reg_type": ["none"],
    "lr_decay_type": ["none"],
}
```

≈ **20k** combination of parameters

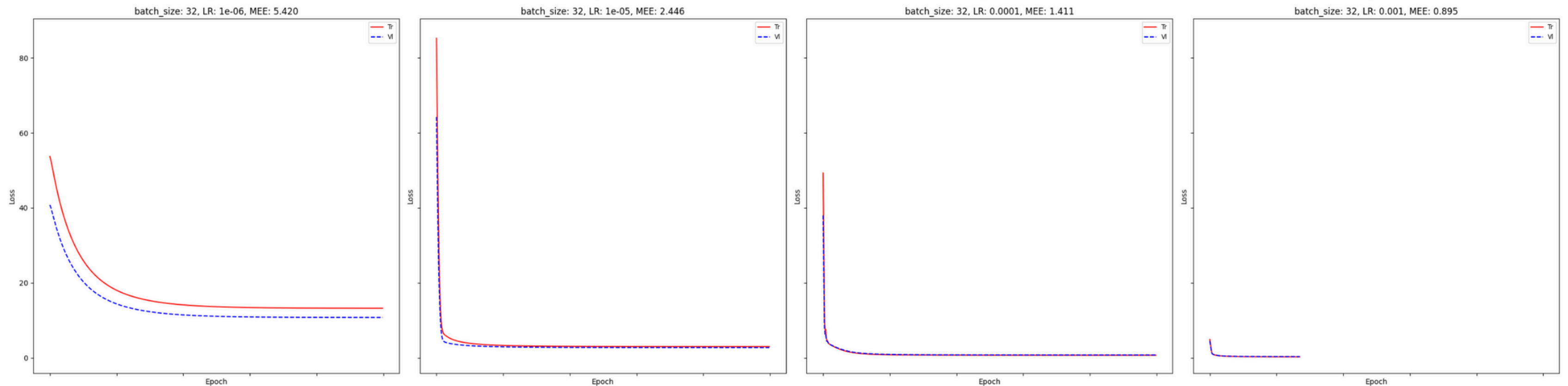
```
param_grid_reg = param_grid_1.copy()
param_grid_reg["lambda_reg"] = [
    1/10**i for i in (np.arange(1,5,1).tolist())
]
param_grid_reg["reg_type"] = ["l1", "l2"]
param_grid_reg['layers'] = [ [17, i, 1] for i in [4, 5] ]
```

≈ **133k** combination of parameters

Appendix: CUP - Screening [*some examples*]



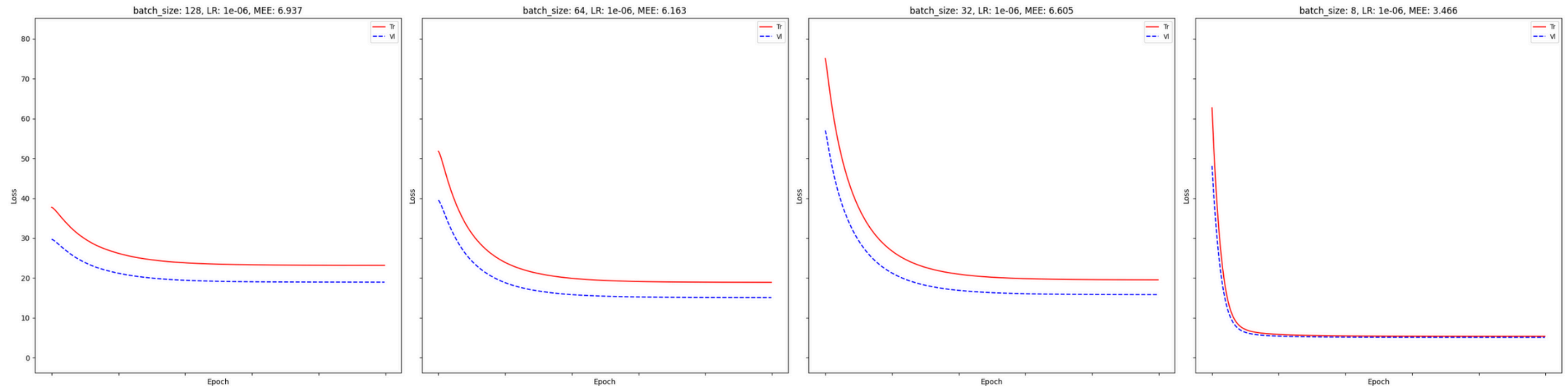
Increasing learning rate keeping batch size (32) fixed => a higher eta is needed to converge.



Appendix: CUP - Screening [*some examples*]



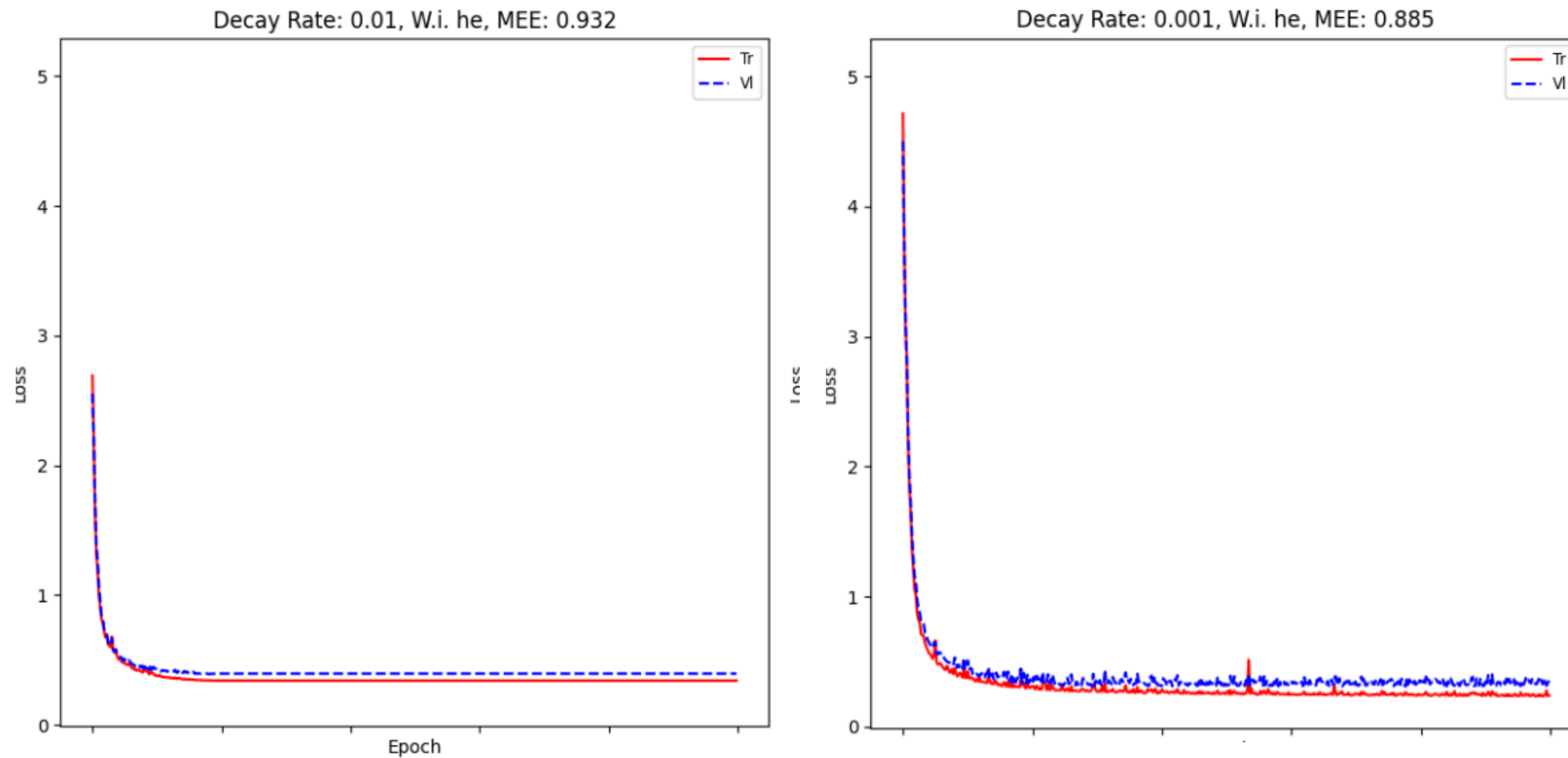
Decreasing batch size keeping learning rate fixed ($1e-6$) => a lower learning rate is more suitable to smaller batch size.



Appendix: CUP - Screening [*some examples*]



Lower decay rate don't reduce the learning rate enough over the epochs, making the curve noisy at the end



Appendix: Score of the params



```
params_count = {
    'learning_rate': {},
    'layers': {},
    'decay_rate': {},
    'lambda_reg': {},
    'momentum_alpha': {},
    'momentum_type': {},
    'weight_init': {},
    'activation_function_names': {},
    'reg_type': {},
    'lr_decay_type': {},
    'batch_size' : {}
}

for idx, param in enumerate(params):
    p = param['params']
    weight = 1 / (idx + 1)

    params_count['learning_rate'][p['learning_rate']] = params_count['learning_rate'].get(p['learning_rate'], 0) + weight
    params_count['layers'][str(p['layers'])] = params_count['layers'].get(str(p['layers']), 0) + weight
    params_count['decay_rate'][p['decay_rate']] = params_count['decay_rate'].get(p['decay_rate'], 0) + weight
    params_count['lambda_reg'][p['lambda_reg']] = params_count['lambda_reg'].get(p['lambda_reg'], 0) + weight
    params_count['momentum_alpha'][p['momentum_alpha']] = params_count['momentum_alpha'].get(p['momentum_alpha'], 0) + weight
    params_count['momentum_type'][p['momentum_type']] = params_count['momentum_type'].get(p['momentum_type'], 0) + weight
    params_count['weight_init'][p['weight_init']] = params_count['weight_init'].get(p['weight_init'], 0) + weight
    params_count['activation_function_names'][str(p['activation_function_names'])] = params_count['activation_function_names'].get(str(p['activation_function_names']), 0) + weight
    params_count['reg_type'][p['reg_type']] = params_count['reg_type'].get(p['reg_type'], 0) + weight
    params_count['lr_decay_type'][p['lr_decay_type']] = params_count['lr_decay_type'].get(p['lr_decay_type'], 0) + weight
    params_count['batch_size'][p['batch_size']] = params_count['batch_size'].get(p['batch_size'], 0) + weight

if SHOW_FULL_OR_TOP_PARAMS == 'top-params':
    for key in params_count:
        total_weight = sum(params_count[key].values())
        for sub_key in params_count[key]:
            params_count[key][sub_key] /= total_weight

print(json.dumps(params_count, indent=4))
```



The end. Thanks for the attention :)